COMPLETE JAVASCRIPT PROGRAMMING SERIES

MODULE 3

CONTROL FLOW & FUNCTIONS

Authors: Mithila Rani Saha & Dithsa Dutta

IT STUDY HUB

www.itstudyhub.dpdns.org

About This Module

Welcome to Volume 3 of the IT Study Hub Complete JavaScript Programming Series. Up to this point, our code has been static—it runs sequentially from line 1 to line 10, executing every single command blindly. In this module, we introduce the concept of "Artificial Intelligence" at its most basic level: Decision Making.

You will learn how to bend time and space in your programs using **Control Flow** (If/Else, Loops) allowing your code to skip lines, repeat actions thousands of times instantly, and make logical choices. We will then wrap up this powerful logic into reusable, modular packages known as **Functions**.

LEARNING OUTCOMES

- Design branching decision trees using `if`, `else if`, `else`, and `switch` statements.
- Automate massive, repetitive tasks using `for`, `while`, and `do...while` loops.
- Precisely control loop execution using the `break` and `continue` keywords.
- Architect reusable code by defining Functions and passing dynamic parameters.
- Adopt modern ES6 features including Arrow Functions and Default Parameters.
- Grasp the foundational security concept of Variable Scope (Global vs Block).

MODULE COVERAGE

Topic	Description
1. Branching Logic	if, else if, else, Nested conditions, switch
2. Iteration (Loops)	for loop, while loop, do...while, break, continue
3. Modularity	Function declarations, Expressions, Arrow functions

IT STUDY HUB

4. Function Mechanics	Parameters, Arguments, Return values, Defaults, Scope

IT STUDY HUB

if

1. CONCEPT

> **The 'if' statement evaluates a condition and executes a block of code only if that condition is true.**

2. SIMPLE EXPLANATION

In real life, we make decisions based on conditions: 'If it is raining, take an umbrella.' In JavaScript, the `if` statement allows your program to make decisions. It checks a logical statement, and if the statement evaluates to `true`, the code inside its curly braces `{}` runs. If it's `false`, the computer completely ignores the code.

3. SYNTAX

```
if (condition) {
// code to run if true
}
```

4. EASY EXAMPLE

Let's check if a user is old enough to vote.

5. CODE

```
let age = 20;

if (age >= 18) {
    console.log("You are eligible to vote!");
}
```

6. OUTPUT

```
Output:
You are eligible to vote!
```

7. CODE EXPLANATION

The engine checks the condition `(age >= 18)`. Since 20 is indeed greater than or equal to 18, it resolves to `true`. Because it is true, the code inside the `{}` executes.

8. REAL-WORLD USE

Checking if a user's password is longer than 8 characters before allowing them to create an account.

9. COMMON MISTAKES

Caution: Putting a semicolon immediately after the parentheses like this: `if (age >= 18); { ... }`. This breaks the connection, and the block will run every time!

10. PRACTICE

Task: Write an `if` statement that logs 'Free shipping!' if a variable `cartTotal` is greater than 50.

else if

1. CONCEPT

> **The 'else if' statement allows you to test multiple specific conditions in a sequence, stopping at the first one that is true.**

2. SIMPLE EXPLANATION

Sometimes you have more than one scenario to check. 'If the light is green, go. Else if the light is yellow, slow down.' The `else if` block acts as a backup question. It is only asked if the first `if` statement was false. You can chain as many `else if` statements together as you need.

3. SYNTAX

```
if (condition1) {
// run if condition1 is true
} else if (condition2) {
// run if condition2 is true
}
```

4. EASY EXAMPLE

Let's grade a student based on their test score.

5. CODE

```
let score = 85;

if (score >= 90) {
    console.log("Grade: A");
} else if (score >= 80) {
    console.log("Grade: B");
} else if (score >= 70) {
    console.log("Grade: C");
}
```

6. OUTPUT

```
Output:
Grade: B
```

7. CODE EXPLANATION

The code evaluates top-to-bottom. Is score >= 90? False. So it skips to the `else if`. Is score >= 80? True! It prints 'Grade: B' and immediately ignores all remaining conditions.

8. REAL-WORLD USE

Displaying different membership badges (Bronze, Silver, Gold, Platinum) based on a user's accumulated points.

9. COMMON MISTAKES

Caution: Checking conditions in the wrong order. If you put `score >= 70` first, a score of 95 would incorrectly receive a 'C' because 95 is greater than 70!

10. PRACTICE

> **Task:** Write an `else if` chain that checks the time of day and says 'Good morning', 'Good afternoon', or 'Good evening'.

else

1. CONCEPT

> **The 'else' statement provides a final, default block of code that runs if all preceding 'if' and 'else if' conditions evaluate to false.**

2. SIMPLE EXPLANATION

The `else` statement is the ultimate fallback plan. It doesn't ask a question; it just says 'If everything else failed, do this.' It is always placed at the very end of an `if` chain and guarantees that *something* will happen, even if all your specific conditions were false.

3. SYNTAX

```
if (condition) {
// ...
} else {
// default code
}
```

4. EASY EXAMPLE

Let's check a user's login status with a default fallback for guests.

5. CODE

```
let isMember = false;

if (isMember === true) {
    console.log("Welcome back, member!");
} else {
    console.log("Please sign up to continue.");
}
```

6. OUTPUT

```
Output:
Please sign up to continue.
```

7. CODE EXPLANATION

Since `isMember` is false, the `if` condition fails. The program automatically falls back to the `else` block to execute the default guest message.

8. REAL-WORLD USE

Charging a default shipping fee of $10 unless the user specifically meets the free-shipping criteria.

9. COMMON MISTAKES

Caution: Trying to add a condition to an `else` block like `else (age < 18) { ... }`. The `else` keyword cannot have parentheses. It catches everything unconditionally.

10. PRACTICE

> **Task:** Extend your `if` statement from the first practice task with an `else` block that says 'Shipping costs $5'.

Nested conditions

1. CONCEPT

> **Nested conditions are simply 'if' statements placed inside other 'if' statements to evaluate complex, multi-step logic.**

2. SIMPLE EXPLANATION

Sometimes, a condition should only be checked if a previous condition has already passed. It's like a locked door behind another locked door. You can't evaluate the inner code unless you first pass the outer requirement. While powerful, nesting too deep makes code incredibly hard to read.

3. SYNTAX

```
if (outerCondition) {
if (innerCondition) {
// runs if BOTH are true
}
}
```

4. EASY EXAMPLE

Let's verify a user has an account, and then verify their password.

5. CODE

```
let userExists = true;
let passwordCorrect = false;

if (userExists) {
    console.log("User found in database.");

    if (passwordCorrect) {
        console.log("Login successful!");
    } else {
        console.log("Incorrect password.");
    }
} else {
    console.log("Account not found.");
}
```

6. OUTPUT

```
Output:
User found in database.
Incorrect password.
```

7. CODE EXPLANATION

The outer `if` confirms `userExists` is true, so it steps inside. Then, it hits a completely new, isolated `if/else` block checking the password. Because the password is wrong, the inner `else` runs.

8. REAL-WORLD USE

> Checking if an item is in stock FIRST, and then checking if the user has enough money in their digital wallet to buy it SECOND.

9. COMMON MISTAKES

> **Caution:** Creating 'Pyramid of Doom' logic (nesting 5 or 6 levels deep). It makes the code visually confusing. Often, combining conditions with `&&` is a better approach.

10. PRACTICE

> **Task: Create a nested `if` that checks if `hasTicket` is true, and inside it, checks if `isVIP` is true.**

switch

1. CONCEPT

> **The 'switch' statement is an alternative control flow tool used to execute different blocks of code based on a single variable matching strict cases.**

2. SIMPLE EXPLANATION

If you have a massive `else if` chain that checks the exact same variable over and over (e.g., checking if the day is Monday, Tuesday, Wednesday...), a `switch` statement is much cleaner. It takes a single value and looks for a matching `case`. Once found, it executes that code. You use the `break` keyword to stop it from running subsequent cases.

3. SYNTAX

```
switch (expression) {
case value1:
// code
break;
default:
```

```
// fallback code
}
```

4. EASY EXAMPLE

Let's output a specific message based on a subscription tier.

5. CODE

```
let role = "admin";

switch (role) {
    case "guest":
        console.log("Limited access.");
        break;
    case "admin":
        console.log("Full control granted.");
        break;
    case "editor":
        console.log("You can edit posts.");
        break;
    default:
        console.log("Role not recognized.");
}
```

6. OUTPUT

```
Output:
Full control granted.
```

7. CODE EXPLANATION

The switch looks at the value of `role` ('admin'). It scans downward until it finds `case "admin":`, executes the console log, and then hits the `break` command, exiting the entire block immediately.

8. REAL-WORLD USE

Handling menu routing in applications or managing keyboard inputs in browser games (checking if the user pressed 'W', 'A', 'S', or 'D').

9. COMMON MISTAKES

Caution: Forgetting the `break` keyword. If you forget it, the switch will experience 'fall-through' and accidentally execute every case below it!

10. PRACTICE

Task: Write a `switch` statement that takes a variable `fruit = 'apple'` and outputs 'Red or Green' for apple, 'Yellow' for banana, and 'Unknown' as default.

for loop

1. CONCEPT

A 'for loop' allows you to repeat a block of code a specific, known number of times.

2. SIMPLE EXPLANATION

Loops automate repetitive tasks. A `for` loop is the most common loop because it keeps all its control mechanics in one tidy line. It consists of three parts separated by semicolons: 1) The Initialization (create a counter), 2) The Condition (keep running while this is true), and 3) The Update (increase the counter after each cycle).

3. SYNTAX

```
for (initialization; condition; update) {
// Code to repeat
}
```

4. EASY EXAMPLE

Let's print the numbers 1 through 5 to the console without writing console.log five times.

5. CODE

```
for (let i = 1; i <= 5; i++) {
    console.log("Iteration number: " + i);
}
```

6. OUTPUT

```
Output:
Iteration number: 1
Iteration number: 2
Iteration number: 3
Iteration number: 4
Iteration number: 5
```

7. CODE EXPLANATION

1) `let i = 1;` starts our counter at 1. 2) The loop checks if `i <= 5`. It is, so the code runs. 3) `i++` increases the counter to 2. It repeats this process until `i` becomes 6, which breaks the loop.

8. REAL-WORLD USE

> Generating 100 empty div containers on a web page, or looping through a known list of exactly 12 months.

9. COMMON MISTAKES

> **Caution:** Using commas `,` instead of semicolons `;` inside the parentheses. Another fatal mistake is creating an infinite loop by forgetting to update the counter (e.g., leaving out `i++`).

10. PRACTICE

> **Task: Write a `for` loop that prints the word 'Hello' 3 times to the console.**

while loop

1. CONCEPT

> **The 'while loop' repeats a block of code continuously as long as a specified condition remains true.**

2. SIMPLE EXPLANATION

While a `for` loop is great when you know EXACTLY how many times to repeat something, a `while` loop is used when you don't know the exact number. It runs indefinitely based entirely on a condition. It asks: 'Is this true? Yes? Run the code.' It then checks again. You must manually ensure that the condition eventually becomes false to stop the loop.

3. SYNTAX

```
while (condition) {
// Code to execute
// Remember to update something to avoid infinite loops
}
```

4. EASY EXAMPLE

Let's simulate draining a battery's power.

5. CODE

```
let battery = 3;

while (battery > 0) {
    console.log("Device is running. Battery: " + battery);
    battery--; // Drain by 1
}

console.log("Device powered off.");
```

6. OUTPUT

```
Output:
Device is running. Battery: 3
Device is running. Battery: 2
Device is running. Battery: 1
Device powered off.
```

7. CODE EXPLANATION

The loop checks `battery > 0`. It runs the code, then `battery--` reduces it to 2. It loops again until `battery` becomes 0. Once 0 > 0 evaluates to false, the loop terminates.

8. REAL-WORLD USE

> Keeping a video game running (`while (gameIsActive)`) or forcing a user to retry a password until they get it right.

9. COMMON MISTAKES

> **Caution:** Creating an Infinite Loop. If we forgot to write `battery--;`, the battery would stay at 3 forever. The loop would run millions of times per second and crash the browser.

10. PRACTICE

> **Task: Create a variable `count = 10`. Write a `while` loop that counts down from 10 to 1, and prints 'Liftoff!' at the end.**

do...while

1. CONCEPT

> **The 'do...while' loop is a variation of the while loop that guarantees the block of code will execute at least once, even if the condition is completely false.**

2. SIMPLE EXPLANATION

In a standard `while` loop, the condition is checked FIRST. If it's false, the code never runs. A `do...while` loop flips this. It executes the code FIRST (the `do` block), and then checks the condition at the end (the `while` block). If the condition is false, it stops, but it still successfully completed one full iteration.

3. SYNTAX

```
do {
// Code runs AT LEAST once
} while (condition);
```

4. EASY EXAMPLE

Let's see what happens when we start with a false condition.

5. CODE

```
let secretCode = false;

do {
    console.log("This prints exactly once, despite being false.");
} while (secretCode === true);

console.log("Loop exited.");
```

6. OUTPUT

```
Output:
This prints exactly once, despite being false.
Loop exited.
```

7. CODE EXPLANATION

The engine blindly enters the `do` block and logs the text. Only at the very end does it check `while(secretCode === true)`. Because it is false, it refuses to do a second loop.

8. REAL-WORLD USE

> Prompting a user with an on-screen popup at least once. It asks 'Do you want to continue?' If they click no, the loop stops, but you guaranteed the popup showed up the first time.

9. COMMON MISTAKES

> **Caution:** Forgetting the semicolon `;` after the `while (condition)` statement, which can sometimes cause syntax parsing issues.

10. PRACTICE

> **Task: Write a `do...while` loop that prints 'Checking system...' and has a condition of `false` so it only runs one time.**

Comparison: for vs while vs do...while

Loop Type	When to use it?	Execution Guarantee
for	When you know exactly how many times you need to loop (e.g., count 1 to 10).	0 or more times.
while	When you don't know the exact count, but rely on a condition changing (e.g., while user is active).	0 or more times.
do...while	When you absolutely must run the code at least once before checking any condition.	**1 or more times.**

break

1. CONCEPT

> **The 'break' keyword acts as an emergency stop button. It immediately terminates a loop or switch statement prematurely.**

2. SIMPLE EXPLANATION

Sometimes you don't need a loop to finish all its cycles. If you are searching a list of 1,000 users for someone named 'Dithsa', and you find her on the 5th loop, it is a massive waste of computer memory to check the remaining 995 users. The `break` command forces JavaScript to instantly smash out of the loop enclosure and move on.

3. SYNTAX

```
if (emergency) {
break;
}
```

4. EASY EXAMPLE

Let's search for a target number and break out once we find it.

5. CODE

```
for (let i = 1; i <= 10; i++) {
    console.log("Checking number " + i);

    if (i === 3) {
        console.log("Target found! Stopping the loop.");
        break; // EMERGENCY STOP
    }
}
console.log("Program continues here...");
```

6. OUTPUT

```
Output:
Checking number 1
Checking number 2
Checking number 3
Target found! Stopping the loop.
Program continues here...
```

7. CODE EXPLANATION

The loop is scheduled to run 10 times. However, on the 3rd iteration, the `if` statement becomes true, triggering the `break`. The loop instantly shatters, ignoring numbers 4 through 10.

8. REAL-WORLD USE

> Stopping a massive data search the millisecond the correct file is located, saving immense CPU power.

9. COMMON MISTAKES

> **Caution:** Putting `break` completely outside of a loop or switch structure. The engine will throw a massive syntax error because it has nothing to break out of.

10. PRACTICE

> **Task: Write a `for` loop from 1 to 5, but use an `if` statement and `break` to stop it when it hits 4.**

continue

1. CONCEPT

> **The 'continue' keyword skips the rest of the current loop iteration and immediately forces the loop to start its next cycle.**

2. SIMPLE EXPLANATION

If `break` is an emergency stop, `continue` is a skip button. Imagine reviewing job applications. If you see an application without a resume, you don't want to stop looking at all the other applicants (`break`); you just want to throw that specific application in the trash and move directly to the next person (`continue`).

3. SYNTAX

```
if (skipCondition) {
continue;
}
```

4. EASY EXAMPLE

Let's print only odd numbers by skipping the even ones.

IT STUDY HUB

5. CODE

```
for (let i = 1; i <= 5; i++) {
    // Check if the number is even (remainder is 0)
    if (i % 2 === 0) {
        continue; // Skip the rest of the code inside this loop
    }

    console.log(i + " is odd.");
}
```

6. OUTPUT

```
Output:
1 is odd.
3 is odd.
5 is odd.
```

7. CODE EXPLANATION

When `i` becomes 2, the `if` statement is true. The `continue` triggers, completely ignoring the `console.log` on line 7, and instantly forcing the loop to jump back to the top, where `i` becomes 3.

8. REAL-WORLD USE

> Filtering out bad data. Looping through 1,000 emails, and using `continue` to skip over the ones marked as spam.

9. COMMON MISTAKES

> **Caution:** Putting code directly beneath a `continue` statement inside an `if` block. That code will become physically impossible to reach.

10. PRACTICE

> **Task:** Write a loop from 1 to 5. Use `continue` to skip printing the number 3.

Functions

1. CONCEPT

> **Functions are reusable blocks of code wrapped up in a package, designed to perform one specific task whenever they are called.**

2. SIMPLE EXPLANATION

In programming, there is a golden rule: DRY (Don't Repeat Yourself). If you find yourself writing the exact same math equation in three different parts of your file, you are doing it wrong. Instead, you write the code once inside a Function, give it a name, and then 'call' that name whenever you need the code executed. It keeps your file small, organized, and clean.

3. SYNTAX

```
function name() {
// code to run
}

name(); // Executing the function
```

4. EASY EXAMPLE

Let's create a machine that prints a standardized greeting.

5. CODE

```
function sayWelcome() {
    console.log("====================");
    console.log("  Welcome to App!   ");
    console.log("====================");
}

// We can 'call' the function as many times as we want
sayWelcome();
sayWelcome();
```

6. OUTPUT

```
Output:
====================
Welcome to App!
====================
====================
Welcome to App!
====================
```

7. CODE EXPLANATION

Lines 1-5 define the function (creating the blueprint). Lines 8 and 9 invoke (execute) the function. The engine jumps back up to line 1, runs the code, and then returns down.

8. REAL-WORLD USE

> Packaging logic like `validateEmail()` or `calculateTax()` so developers can reuse it across hundreds of different webpages.

9. COMMON MISTAKES

> **Caution:** Forgetting the parentheses `()` when trying to run the function. If you write `sayWelcome;`, JavaScript will just look at the blueprint, it won't actually execute it.

10. PRACTICE

> **Task: Define a function named `sayGoodbye` that logs 'See you later!' and call it once.**

Function Declarations

1. CONCEPT

> **A Function Declaration is the traditional, standard way to define a function, which benefits from JavaScript's 'Hoisting' mechanism.**

2. SIMPLE EXPLANATION

When you start a line specifically with the keyword `function`, it is a declaration. The unique superpower of function declarations is that they are 'Hoisted'. Before your code even runs, the JavaScript engine scans the entire document, finds all Function Declarations, and lifts them to the very top of memory. This means you can execute the function on line 1, even if you define it on line 100!

3. SYNTAX

```
function calculate() {
// body
}
```

4. EASY EXAMPLE

Let's observe the magic of hoisting.

5. CODE

```
// 1. Executing the function BEFORE it is created!
startEngine();

// 2. Defining the function down here
function startEngine() {
    console.log("Vroom! Engine is running.");
}
```

6. OUTPUT

```
Output:
Vroom! Engine is running.
```

7. CODE EXPLANATION

Even though `startEngine()` is called on line 2, the code does not crash. During the invisible preparation phase, the JavaScript engine hoisted the entire `startEngine` block into memory first.

8. REAL-WORLD USE

Many developers put all their execution code at the top of a file, and hide hundreds of lines of function declarations at the bottom to keep the file visually clean.

9. COMMON MISTAKES

Caution: Overusing massive function declarations scattered chaotically through a file, making it hard to track where logic originates.

10. PRACTICE

> **Task:** Call a function named `greetWorld()` on line 1, and write its function declaration on line 3.

Function Expressions

1. CONCEPT

> **A Function Expression involves creating an anonymous (nameless) function and assigning it directly to a variable.**

2. SIMPLE EXPLANATION

In JavaScript, functions are treated just like any other data type (Strings, Numbers, Booleans). Because they are data, you can shove an entire function inside a standard variable box (`let`). The major difference here is that variables are NOT fully hoisted like declarations. You MUST define the expression before you can use it.

3. SYNTAX

```
let myFunc = function() {
// body
};
```

4. EASY EXAMPLE

Let's see what happens if we try to hoist a function expression.

5. CODE

```
let walk = function() {
    console.log("Walking step by step.");
};

walk(); // This works perfectly.

// ❌ UNCOMMENTING BELOW WILL CRASH
// run();
// let run = function() { console.log("Running!"); };
```

6. OUTPUT

```
Output:
Walking step by step.
```

7. CODE EXPLANATION

The `walk` function executes flawlessly because it was defined before it was called. If you attempt to call `run()` before it is defined, JavaScript throws a fatal error because the variable `let run` exists in the Temporal Dead Zone.

8. REAL-WORLD USE

Passing functions around as data (callbacks), commonly used in event listeners (e.g., `button.onclick = function() { ... }`).

9. COMMON MISTAKES

Caution: Trying to use function expressions before defining them, which is the most common bug when transitioning away from declarations.

10. PRACTICE

> **Task:** Create a function expression assigned to a `const` named `laugh` that logs 'Hahaha'.

Parameters and Arguments

1. CONCEPT

> **Parameters are placeholders used to pass external data into a function. Arguments are the actual, real values passed in.**

2. SIMPLE EXPLANATION

If a function is a juice blender, a Parameter is the empty slot on top where fruit goes. An Argument is the actual, physical apple or banana you drop in. This makes functions incredibly dynamic. Instead of a function that only calculates `5 + 5`, you can create a function that takes two generic parameters (`x` and `y`) and calculates whatever the user feeds it.

3. SYNTAX

```
// x and y are parameters (placeholders)
function add(x, y) { }

// 5 and 10 are arguments (real data)
add(5, 10);
```

4. EASY EXAMPLE

Let's build a personalized greeting machine.

5. CODE

```
// 'userName' and 'time' are parameters
function greetUser(userName, time) {
    console.log(`Good ${time}, ${userName}!`);
}

// 'Mithila' and 'morning' are the arguments being injected
greetUser("Mithila", "morning");

// We can reuse the same blueprint with entirely new data!
greetUser("Dithsa", "evening");
```

6. OUTPUT

```
Output:
Good morning, Mithila!
Good evening, Dithsa!
```

7. CODE EXPLANATION

When `greetUser("Mithila", "morning")` is called, JavaScript takes the string 'Mithila' and temporarily stores it inside the variable `userName`. It does the same for `time`, allowing the template literal to print a custom sentence.

8. REAL-WORLD USE

> A `calculateTotal(price, tax)` function that calculates the final cart cost based on varying prices and state tax laws.

9. COMMON MISTAKES

> **Caution:** Passing the wrong number of arguments. If you define two parameters but only pass one argument, the second parameter becomes `undefined`, which usually breaks math calculations resulting in `NaN`.

10. PRACTICE

> **Task: Create a function `multiply(a, b)` that console logs the result of `a * b`. Test it with the numbers 5 and 4.**

Return Values

1. CONCEPT

> **The 'return' keyword pushes a final calculated value OUT of the function, handing it back to the program, and instantly stops the function.**

2. SIMPLE EXPLANATION

Up to now, our functions just printed text to the console. But `console.log` doesn't actually give data back to the computer—it just displays it for humans to read. If you want a function to do heavy math and then hand that answer back to a variable so you can use it elsewhere, you must use the `return` keyword.

3. SYNTAX

```
function getSum(a, b) {
return a + b;
```

```
}
let result = getSum(2, 2);
```

4. EASY EXAMPLE

Let's capture the result of a function.

5. CODE

```
function calculateSquare(num) {
    let answer = num * num;
    return answer; // Push data OUT of the function
    console.log("This will NEVER print.");
}

// The function becomes the value 25, which gets stored in box
let myFinalBox = calculateSquare(5);

console.log("Stored in memory:", myFinalBox);
```

6. OUTPUT

```
Output:
Stored in memory: 25
```

7. CODE EXPLANATION

The function calculates 5 * 5. The `return` statement hands `25` back to the global scope where it is caught by `myFinalBox`. Notice the console log inside the function is ignored; `return` instantly shreds the function and stops execution.

8. REAL-WORLD USE

Fetching user data from a server and `return`ing it to the screen so it can be formatted into an HTML profile card.

9. COMMON MISTAKES

Caution: Forgetting to write `return`. If you forget it, the function does the math successfully, but hands back `undefined` by default, destroying your data.

10. PRACTICE

Task: Write a function `double(x)` that returns `x * 2`. Store the output in a variable and print it.

Default Parameters

1. CONCEPT

Default parameters allow you to pre-assign fallback values to parameters in case the developer forgets to pass an argument.

2. SIMPLE EXPLANATION

Introduced in modern ES6 JavaScript, default parameters are a safety net. If a function requires two pieces of data, but the user only provides one, the missing piece normally becomes `undefined`, causing errors. By assigning a default value directly in the parentheses with an `=`, JavaScript will automatically use that fallback value if data is missing.

3. SYNTAX

```
function greet(name = 'Guest') {
// ...
}
```

4. EASY EXAMPLE

Let's secure our greeting function against missing data.

5. CODE

```
function setupAccount(username = "Anonymous", role = "User") {
    console.log(`Created ${role} account for ${username}.`);
}

// 1. Providing ALL arguments perfectly
setupAccount("AdminMithila", "SuperAdmin");

// 2. Providing only ONE argument (role defaults)
setupAccount("Dithsa");

// 3. Forgetting EVERYTHING (both default)
setupAccount();
```

6. OUTPUT

```
Output:
Created SuperAdmin account for AdminMithila.
Created User account for Dithsa.
Created User account for Anonymous.
```

7. CODE EXPLANATION

When we pass missing data in calls 2 and 3, JavaScript realizes an argument is missing. Instead of throwing an `undefined` error, it smoothly falls back to the `=` defaults we set in the blueprint.

8. REAL-WORLD USE

> Creating a `createButton(color = 'blue', size = 'medium')` function so developers don't have to manually specify generic styling every time.

9. COMMON MISTAKES

> **Caution:** Putting default parameters first. Always put parameters with defaults at the END of the list. E.g., `function(a, b = 2)`. If you do `function(a = 2, b)`, argument mapping becomes chaotic.

10. PRACTICE

> **Task: Write a function `rollDice(sides = 6)`. Log 'Rolling a [sides]-sided dice.' Call it with no arguments.**

Arrow Functions

1. CONCEPT

> **Arrow functions provide a modern, ultra-compact syntax for writing function expressions using a fat arrow '=>'.**

2. SIMPLE EXPLANATION

Developers love writing less code. In 2015, ES6 introduced Arrow Functions. They strip away the `function` keyword, allow you to drop the `return` keyword for single-line math, and remove `{}` brackets. They are simply a faster, cleaner way to write function expressions.

3. SYNTAX

```
let func = (params) => { /* body */ };

// One-line implicit return
let func = a => a * 2;
```

4. EASY EXAMPLE

Let's compress a massive function into a single, elegant line of code.

5. CODE

```
// Old, bulky ES5 way:
let oldAdd = function(a, b) {
    return a + b;
};

// Modern, sleek ES6 Arrow Function:
const newAdd = (a, b) => a + b;

console.log(oldAdd(10, 5));
console.log(newAdd(10, 5));
```

6. OUTPUT

```
Output:
15
15
```

7. CODE EXPLANATION

The `newAdd` variable holds an arrow function. Because the logic is only one line long, we delete the curly braces `{}` and the `return` word. The fat arrow `=>` automatically implies a return. Both functions do the exact same thing.

8. REAL-WORLD USE

> Used heavily in React.js and when manipulating large Arrays using array methods like `.map()` and `.filter()`.

9. COMMON MISTAKES

> **Caution:** Trying to use an Arrow Function as an Object Method. Arrow functions do not have their own internal `this` context, which causes severe bugs in object-oriented programming.

10. PRACTICE

> **Task: Convert `let subtract = function(x, y) { return x - y; }` into a one-line arrow function.**

Comparison: Function Declarations vs Expressions vs Arrow Functions

Type	Syntax Example	Hoisted?	Best Used For
Declaration	`function x() {}`	Yes	Large, core application logic that needs to be accessed anywhere.
Expression	`let x = function() {}`	No	Callbacks and restricting access to variables.
Arrow		No	

	`const x = () => {}`		Short math, clean inline logic, and modern frontend frameworks.

Scope Basics

1. CONCEPT

> **Scope determines the visibility and accessibility of variables in different parts of your code.**

2. SIMPLE EXPLANATION

If a variable is a physical item, Scope is the room it lives in. JavaScript has 'Global Scope' (the main house) and 'Block Scope' (a locked safe inside the house). Variables created outside functions are Global—anyone can see them. Variables created INSIDE a function or block `{}` using `let` or `const` are trapped inside. The outside world cannot see them.

3. SYNTAX

```
{ let trapped = true; }
// trapped is dead here
```

4. EASY EXAMPLE

Let's try to access a trapped variable.

5. CODE

```
let globalName = "World"; // Visible everywhere

function doMagic() {
    let secretWord = "Abracadabra"; // Trapped inside the function!
    console.log("Inside function:", globalName); // Can see outside
    console.log("Inside function:", secretWord); // Can see inside
}

doMagic();
// ❌ THIS WILL CRASH:
// console.log(secretWord);
```

6. OUTPUT

```
Output:
Inside function: World
Inside function: Abracadabra
(Error if last line is uncommented: secretWord is not defined)
```

7. CODE EXPLANATION

The function can look 'up' and see `globalName`. But the global space cannot look 'down' into the function to see `secretWord`. Once the function finishes running, `secretWord` is physically destroyed in memory.

8. REAL-WORLD USE

> Scope acts as a security system. By keeping sensitive variables inside specific functions, hackers and other developers cannot accidentally overwrite your data from the outside.

9. COMMON MISTAKES

Caution: Declaring variables inside a loop, and then trying to use the final result of that variable outside the loop. It will crash because the variable was trapped in the loop's block scope.

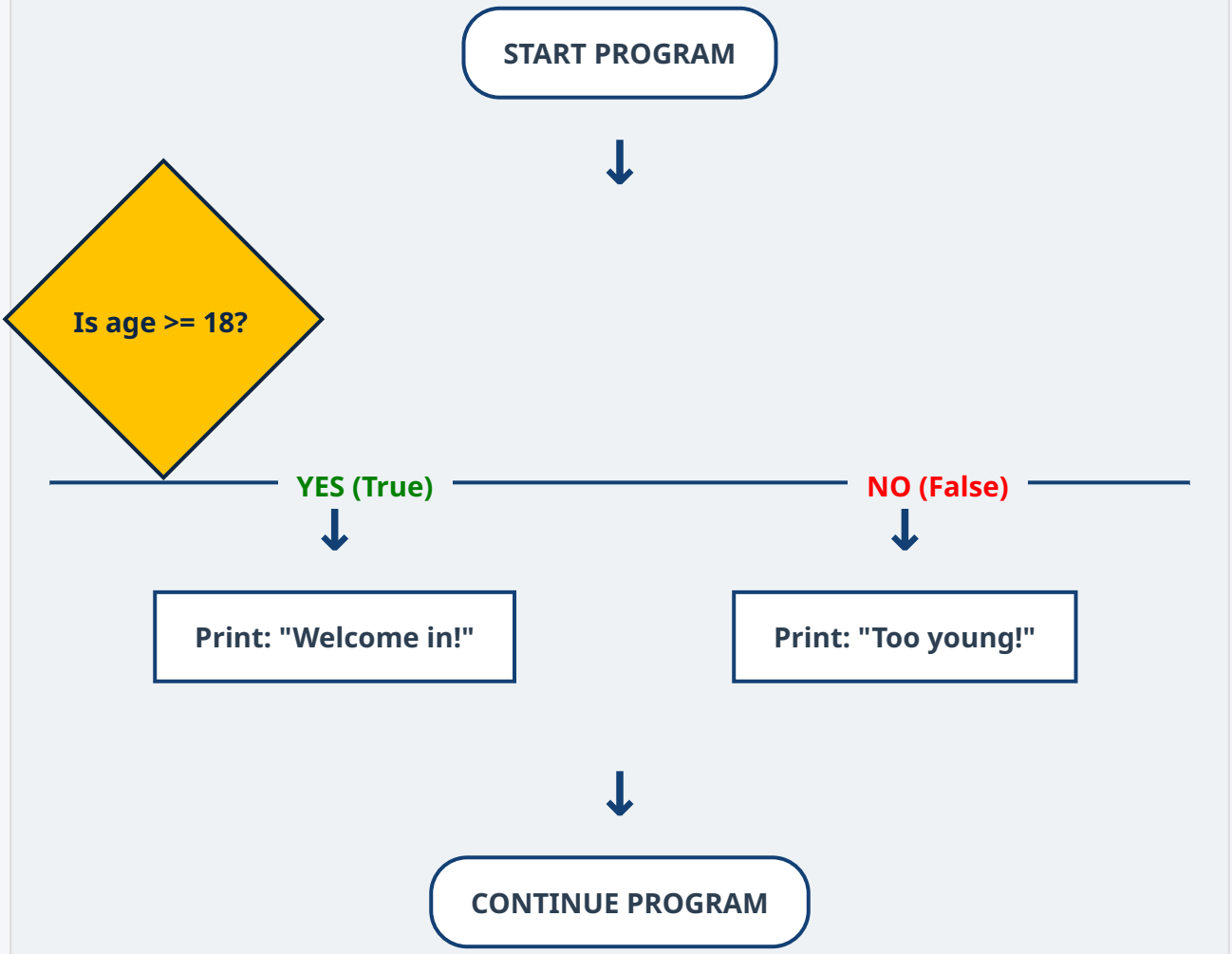
10. PRACTICE

Task: Create an `if(true) { let secret = 42; }` block. Try to `console.log(secret)` outside the block and observe the crash.

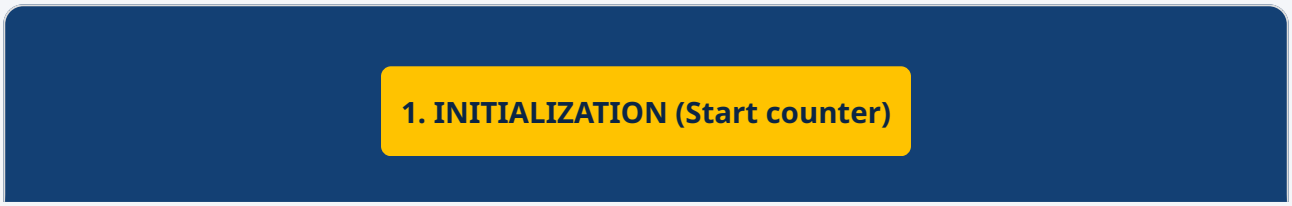
IT STUDY HUB

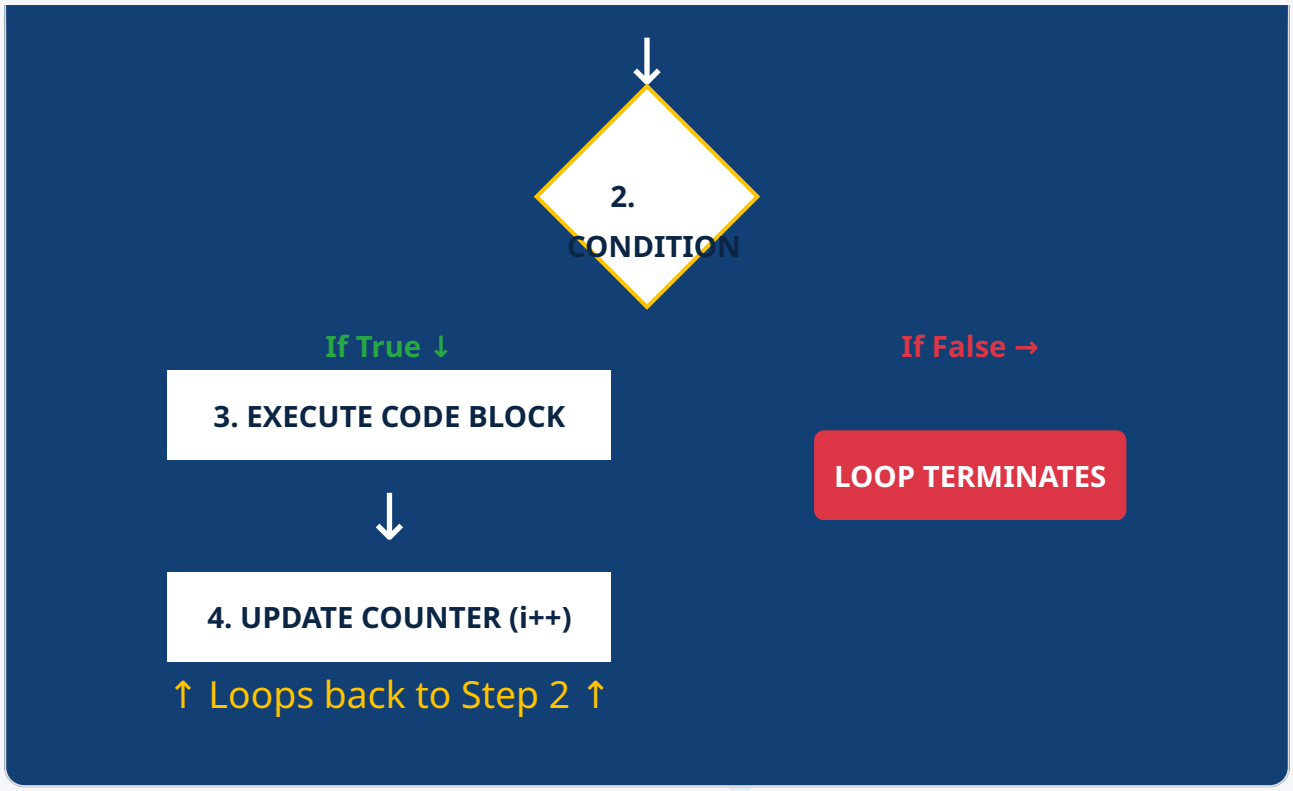
Visual Architecture & Workflows

1. IF/ELSE DECISION FLOWCHART



2. THE LOOP EXECUTION CYCLE (FOR / WHILE)




2. CONDITION
If True ↓
If False →
3. EXECUTE CODE BLOCK
LOOP TERMINATES
4. UPDATE COUNTER (i++)
↑ Loops back to Step 2 ↑

IT STUDY

Hands-on Project: Number Guessing Game

We will combine loops, if/else statements, and functions to create a fully interactive game. The computer will pick a random number between 1 and 10, and the user must guess it using HTML inputs.

STEP 1: THE STRUCTURE (INDEX.HTML)

Create a simple interface allowing the user to type a guess and click a button.

IT STUDY HUB

```
<!DOCTYPE html>
<html>
<head>
    <title>Number Guessing Game</title>
    <style>
        body { font-family: Arial; padding: 50px; text-align: center;
background: #eef2f5; }
        .card { background: white; padding: 30px; border-radius: 10px; box-
shadow: 0 4px 8px rgba(0,0,0,0.1); display: inline-block;}
        input { padding: 10px; margin: 10px; border: 2px solid #134074;
font-size: 16px; }
        button { padding: 10px 20px; background: #134074; color: white;
border: none; cursor: pointer; font-size: 16px; }
        #message { margin-top: 20px; font-weight: bold; font-size: 1.2em;
color: #0B2545;}
    </style>
</head>
<body>
    <div class="card">
        <h2>Guess the Secret Number! (1-10)</h2>
        <input type="number" id="guessInput" placeholder="Enter your guess">
        <br>
        <button onclick="checkGuess()">Submit Guess</button>
        <div id="message"></div>
    </div>
    <script src="app.js"></script>
</body>
</html>
```

STEP 2: THE LOGIC (APP.JS)

We generate the secret number exactly once when the file loads, and then write a function that executes *every time* the user clicks the submit button.

```
"use strict";

// 1. Generate the random secret number globally (Scope)
// Math.random() generates a decimal. We multiply by 10, round down, and add
1.
const secretNumber = Math.floor(Math.random() * 10) + 1;

// 2. Track how many attempts the user has made
let attempts = 0;

// 3. The reusable Function declaration
function checkGuess() {
    attempts++; // Increment attempts each time they click

    let rawInput = document.getElementById('guessInput').value;
    let userGuess = Number(rawInput);
    let messageBox = document.getElementById('message');

    // 4. Branching Logic (If / Else If / Else)
    if (userGuess === secretNumber) {
        messageBox.style.color = "green";
        messageBox.innerHTML = `🎉 Correct! You won in ${attempts} attempts!
`;

    } else if (userGuess > secretNumber) {
        messageBox.innerHTML = "❌ Too high! Try again.";

    } else if (userGuess < secretNumber) {
        messageBox.innerHTML = "❌ Too low! Try again.";

    } else {
        // Fallback catch in case they type something completely invalid
        messageBox.innerHTML = "⚠️ Please enter a valid number.";
    }
}
```

Code Explanation:

- **Scope:** `secretNumber` and `attempts` are defined OUTSIDE the function (Global). If we put

them inside, the secret number would completely randomize itself every time you clicked the button!
- **Math.random():** A built-in JS tool. It handles the random generation.
- **else if:** The function perfectly filters the user's input, giving them helpful feedback if they are too high or too low, guiding them to the win state.

EXTENSION CHALLENGES FOR PRACTICE:

- Add a `while` loop that forces a max limit of 3 attempts. If they fail 3 times, lock the input box and show 'GAME OVER'.
- Add a 'Reset Game' button that re-rolls the `secretNumber` and sets `attempts` back to 0 without refreshing the webpage.

IT STUDY HUB

Module 3 Assessment & Practice

Knowledge is solidified through repetition. Complete these sections to master the module.

15 BEGINNER EXERCISES

1. **Write an `if` statement checking if `temperature > 30` to log 'It is hot!'.**

2. **Chain an `else if` to check if `temperature < 15`, logging 'It is cold!'.**

3. **Add a final `else` block logging 'The weather is mild.'**

4. **Refactor a simple `if/else` block checking admin status into a Ternary Operator.**

5. **Write a `switch` statement checking a variable `status`. Handle 'loading', 'success', and 'error' cases.**

6. **Create a `for` loop that prints numbers from 1 to 10.**

7. **Create a `for` loop that prints numbers from 10 down to 1.**

8. **Write a `while` loop that doubles a variable `x = 2` until it is greater than 100.**

9. **Use a `break` in a loop from 1 to 20 to stop exactly when `i` reaches 13.**

IT STUDY HUB

10. Write a function declaration `greet(name)` that prints 'Hello, [name]'.

11. Rewrite the `greet` function as a function expression assigned to a `const`.

12. Rewrite the `greet` function once more as an Arrow Function.

13. Write a function `add(a, b)` that explicitly uses the `return` keyword.

14. Give parameter `b` in the previous function a default value of `10`.

15. Explain in simple terms why a variable declared inside a function cannot be accessed outside.

IT STUDY HUB

20 VIVA QUESTIONS & ANSWERS

Question	Answer
What happens if you omit the `break` statement in a `switch` case?	The switch experiences 'fall-through', executing all subsequent cases sequentially until it hits a break or the end.
What is the difference between an `if/else` block and a ternary operator?	They do the same thing, but a ternary operator is an expression that returns a value in a single line, making it cleaner for simple logic.
In a `for` loop, which part executes first: initialization or condition?	The initialization executes first (and only once).
What is an infinite loop?	A loop where the condition never evaluates to false, causing the program to run forever and crash the browser.
When is a `do...while` loop preferable over a `while` loop?	When you need the internal code to execute at least one time regardless of the condition.
What is the specific purpose of the `continue` keyword?	It skips the remaining code in the current iteration and forces the loop to start the next iteration immediately.
What is Hoisting?	JavaScript's invisible behavior of moving function declarations and variable declarations to the top of their scope before execution.
Does hoisting apply to Function Expressions?	No, only standard Function Declarations are hoisted fully with their logic.

What is the difference between a parameter and an argument?	Parameters are the placeholders defined in the function blueprint; arguments are the actual data passed in during execution.
What value does a function return if there is no explicit `return` keyword?	It returns `undefined`.
What happens to code written immediately after a `return` statement inside a function?	It becomes unreachable code. The `return` keyword instantly terminates the function execution.
What is block scope?	The rule that variables declared with `let` or `const` inside `{}` (curly braces) are trapped and invisible to the outside code.
Can you define default parameters in ES5 (the old way)?	No, you had to manually check if the parameter was undefined and reassign it. Default parameters (`x = 5`) were introduced in ES6.
What syntax feature makes Arrow functions highly preferred for simple math?	The implicit return. You can drop the `{}` and `return` keyword for one-line logic.
What is a 'Callback Function'?	A function that is passed as an argument directly into another function to be executed later.
Can you use the `break` keyword inside an `if` statement that is NOT inside a loop?	No. `break` is strictly reserved for loops and switch statements. Doing so throws a Syntax Error.
If I declare `var x = 10` inside an `if` block, is it block-scoped?	No. `var` is function-scoped, meaning it 'leaks' out of the `if` block. This is why `var` is heavily discouraged today.

What happens if a function requires 3 arguments but you only pass 2?	The 3rd parameter receives the value `undefined`, unless a default parameter was explicitly set.
What is a Global Variable?	A variable declared completely outside of any functions or blocks, making it accessible and editable by any part of the program.
Why is it dangerous to rely on Global Variables?	Any function can accidentally overwrite a global variable, causing unpredictable bugs across massive applications.

IT STUDY HUB

10 INTERVIEW QUESTIONS

These questions test your deeper conceptual understanding. Practice answering them verbally.

- **Explain the internal execution mechanics of a `switch` statement vs an `else if` chain regarding strict equality (===).**
- **Walk me through a scenario where utilizing a `do...while` loop is the only logical choice over a `for` loop.**
- **Describe the 'Temporal Dead Zone' in relation to scoping and function expressions.**
- **How do Arrow Functions handle the `this` keyword differently compared to traditional Function Declarations? (Conceptual Preview)**
- **Explain how combining the `break` keyword with 'labeled statements' can control deeply nested loops.**
- **If you have a highly complex, 20-line math equation, would you use an implicit-return Arrow Function? Why or why not?**
- **Demonstrate how 'Lexical Scoping' works when a nested function tries to access variables from its parent function.**
- **Why is 'Don't Repeat Yourself' (DRY) a critical principle, and how do functions enforce it at an architectural level?**
- **What are 'Pure Functions', and why are they considered safer and more predictable than standard functions that modify global variables?**
- **Write down the process of converting an ES5 'arguments' object approach to modern ES6 Default Parameters.**

IT STUDY HUB

Module Summary

In this module, you transformed from a passive scriptwriter into an active logic programmer. You learned how to direct the flow of your program's execution using decision trees (`if`, `else if`, `else`, `switch`). You conquered time by automating highly repetitive actions using loops (`for`, `while`, `do...while`) and utilized strict safety stops like `break` and `continue` to keep those loops optimized.

Most importantly, you leveled up your software architecture by packaging code into **Functions**. You understand the profound difference between a traditional hoisted Function Declaration, a tightly-bound Function Expression, and a sleek modern Arrow Function. You can now pass dynamic parameters, capture computed returns, and safely isolate your sensitive data using strict Block Scope.

WHAT COMES NEXT?

MODULE 4 — ARRAYS, OBJECTS & STRINGS

We have the logic, we have the math, and we have the engine. Now, we need to handle mass amounts of data. In Module 4, you will learn how to manipulate colossal Data Structures. We will dissect the magic of Arrays (ordered lists), dive deeper into complex Objects, and master built-in methods designed to sort, filter, and manipulate vast strings of text instantly.